



SIMATS
ENGINEERING



SIMATS

Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

IMPORTANCE OF SDLC IN E-COMMERCE DEVELOPMENT

Object Oriented Analysis and Design

A structured engineering approach to building reliable, secure, and customer-ready E-Commerce platforms

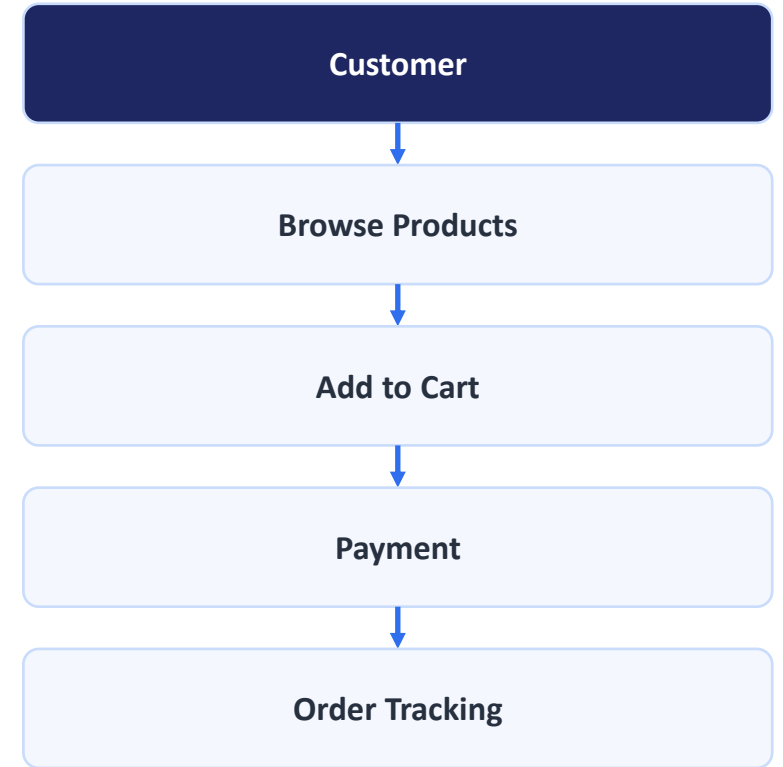


Introduction

Understanding E-Commerce and why software quality is critical

- An **E-Commerce platform** is software that lets customers browse, buy, and pay for products online.
- Core features include **Product Management, Online Payments, and Order Tracking.**
- Customers expect the platform to be **fast, secure, and always available.**
- A single bug in checkout can mean **lost sales and lost trust.**
- Software quality directly decides whether customers **return or leave.**

A Simple Customer Journey



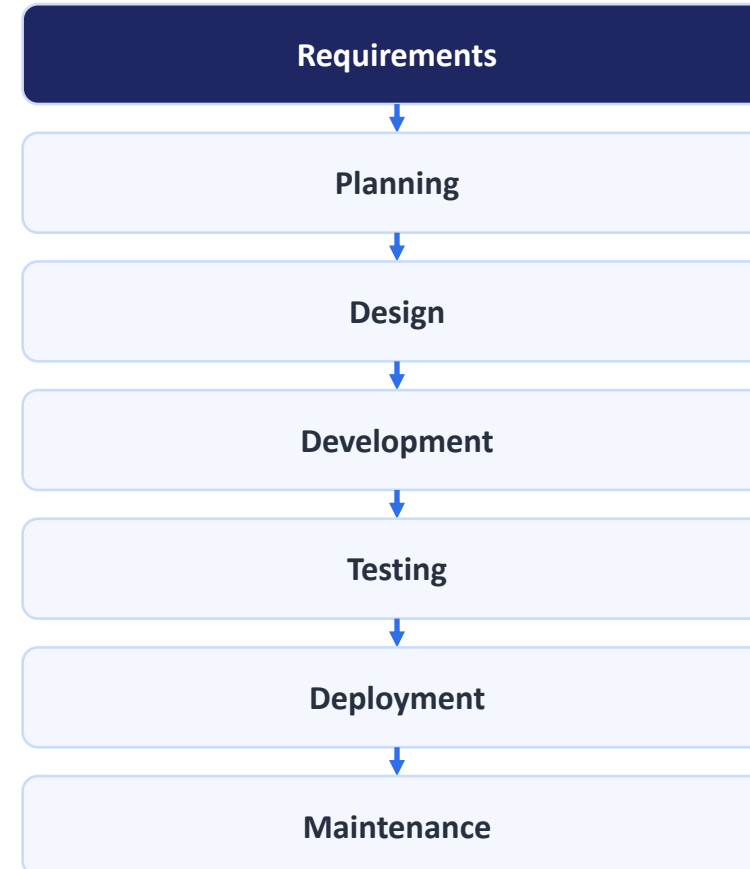


What is SDLC?

A defined sequence of phases for building software correctly

- **SDLC (Software Development Life Cycle)** is a structured set of phases used to plan, build, test, and maintain software.
- Each phase has a clear **goal, input, and output** before the next begins.
- It converts a vague idea like "sell products online" into a **working, reliable system**.
- For E-Commerce, SDLC ensures **quality, security, and scalability** are built in from day one — not patched later.

The SDLC Lifecycle



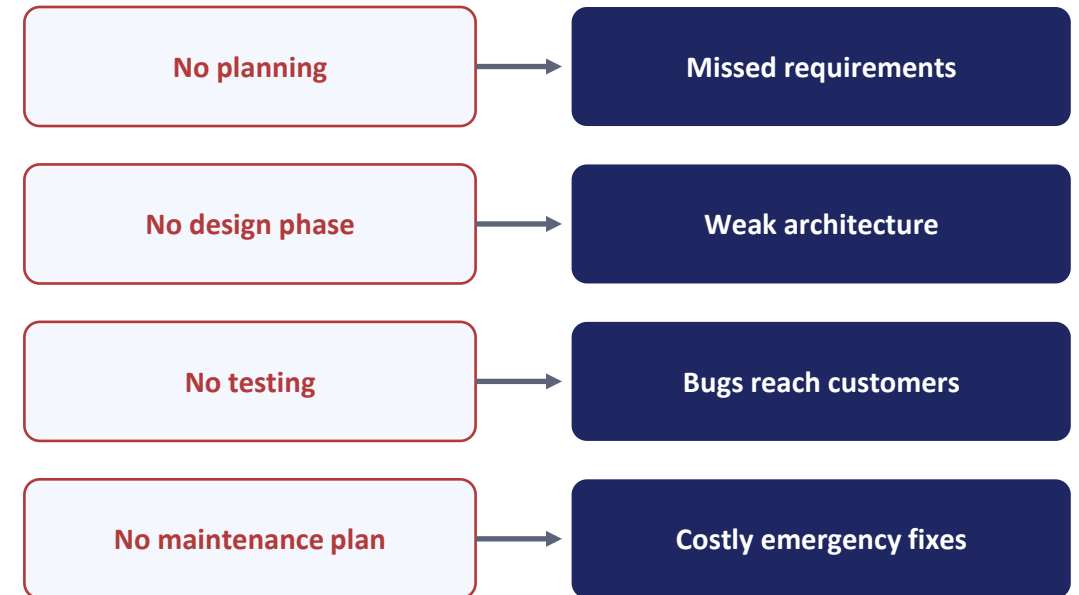


Why Do We Need SDLC?

What goes wrong when E-Commerce is built without a process

- Without a process, teams **skip planning** and jump straight to coding.
- This leads to **more bugs** discovered late, when they are costly to fix.
- **Security issues** slip through, exposing customer data.
- **Payment failures** occur because edge cases were never analysed.
- Fixing problems after launch causes **high maintenance cost**.

Cause → Effect





Requirement Analysis & Planning

Deciding exactly what the platform must do, before writing code

- **Requirement Gathering:** list every feature the platform needs
 - Login, Product Search, Cart, Payments, Order Tracking.
- **Planning:** estimate timeline, budget, team roles, and required technology.
- Ambiguity here (e.g. "fast checkout") is clarified into **measurable goals** (e.g. under 3 seconds).
- A missed requirement now means a **redesign later** — this phase prevents that.

E-Commerce Requirements



Login



Product Search



Cart



Payments



Order Tracking

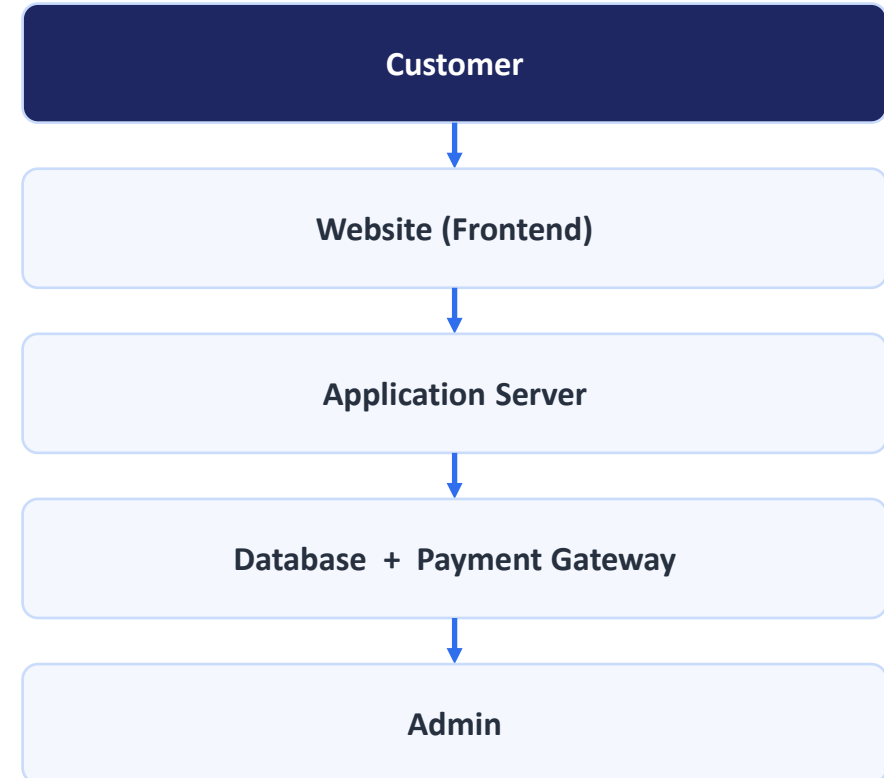


System Design

Architecting how the platform will be structured internally

- **UI Design:** how pages look and how customers navigate — product pages, cart, checkout.
- **Database Design:** how product, user, and order data is organised and related.
- **Architecture Design:** how the website, servers, database, and payment gateway connect and communicate.
- Good design here prevents **bottlenecks and security gaps** during high-traffic sales.

System Architecture



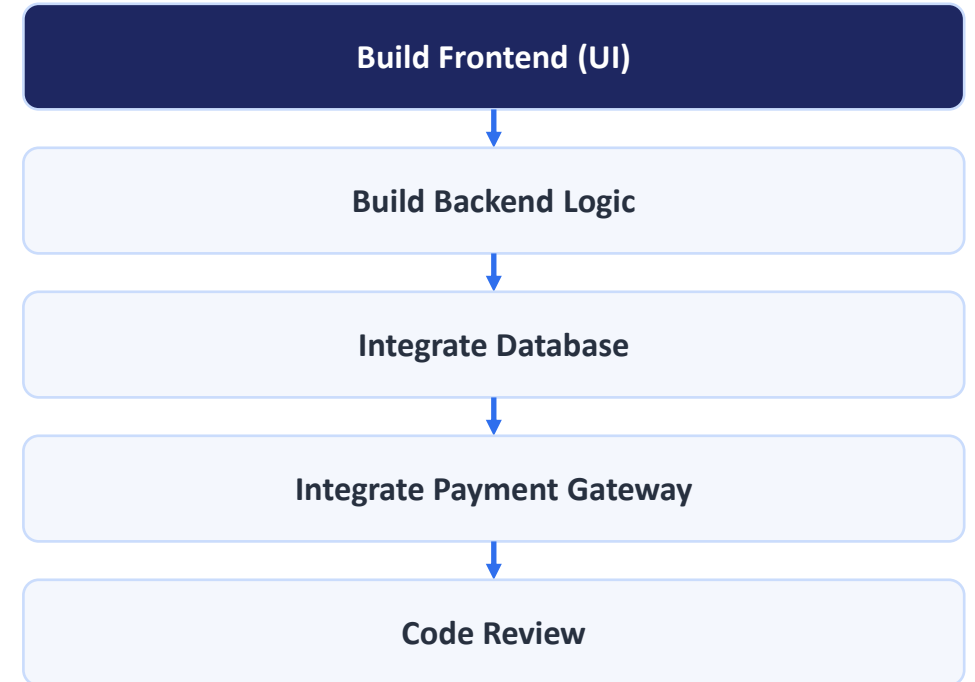


Development

Turning the design into working code

- **Frontend:** builds the customer-facing pages — product listings, cart, checkout screens.
- **Backend:** implements business logic — pricing, discounts, order rules, inventory checks.
- **Database Integration:** connects the application to product, user, and order data.
- **Payment Integration:** securely connects to a payment gateway (e.g. UPI, cards, wallets).
- Following the design from Phase 3 keeps development **consistent and bug-resistant**.

Development Workflow



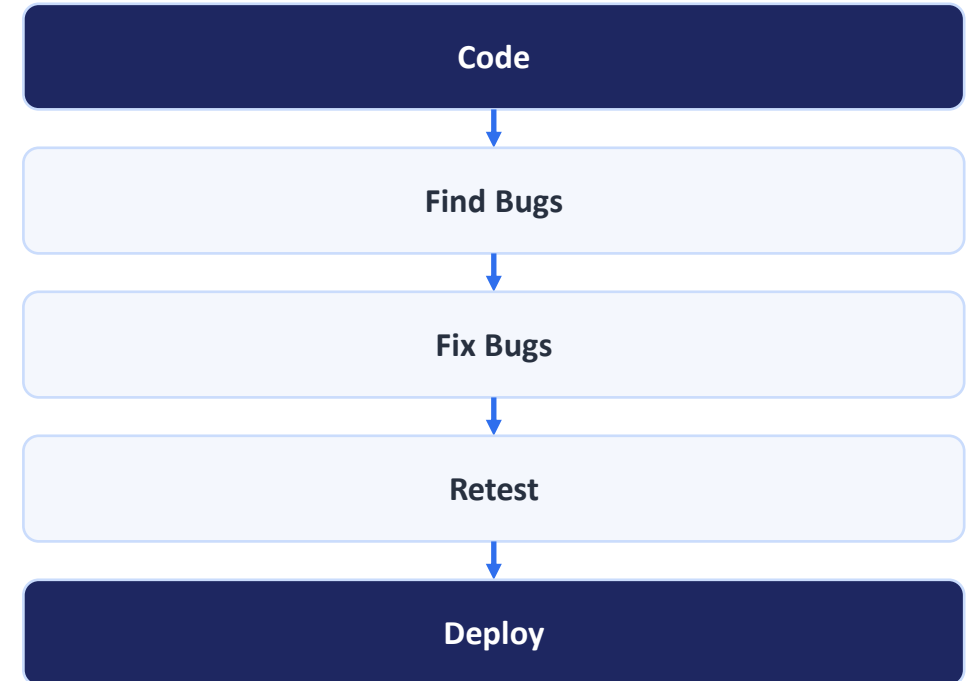


Testing

Verifying the platform works correctly before customers see it

- **Unit Testing:** checks individual functions, like discount calculation.
- **Integration Testing:** checks that cart, payment, and order modules work together.
- **System Testing:** checks the full purchase flow end-to-end.
- **Security Testing:** checks that customer and payment data cannot be stolen.
- **Performance Testing:** checks the site stays fast during sales traffic spikes.

Testing Workflow



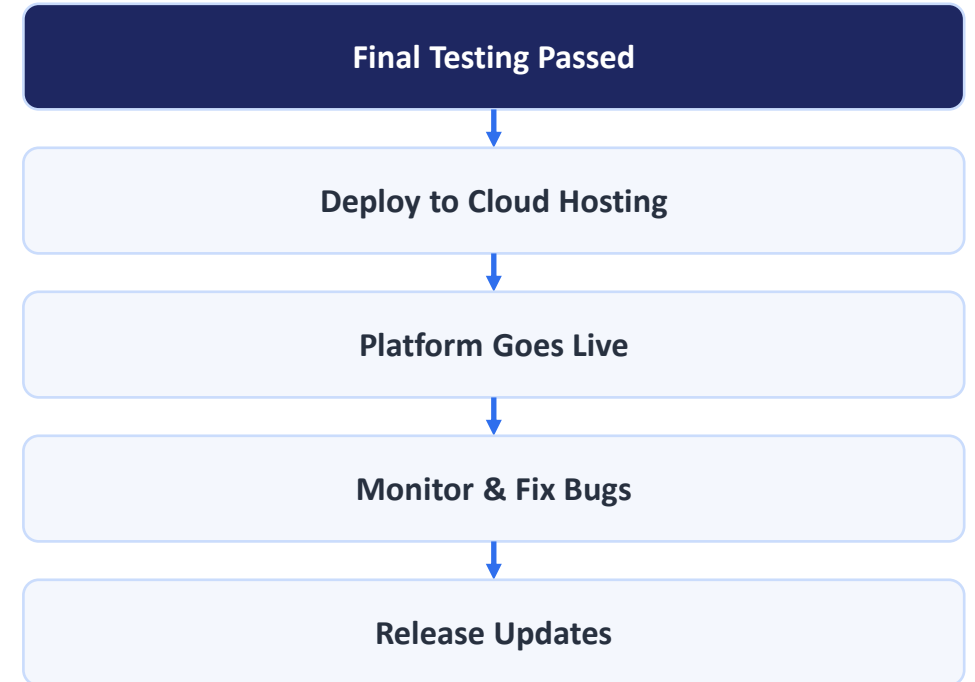


Deployment & Maintenance

Launching the platform and keeping it healthy afterward

- **Deployment — Launch:** the platform goes live for real customers.
- **Deployment — Cloud Hosting:** hosting on the cloud allows the site to scale during demand spikes.
- **Maintenance — Bug Fixes:** issues found in production are resolved quickly.
- **Maintenance — Security Updates:** patches protect against new threats.
- **Maintenance — Feature Enhancements:** new features are added based on customer feedback.

Deployment Flow





Analysis: How Each Phase Adds Value

Connecting every SDLC phase to real business outcomes

SDLC Phase	Software Quality	Reliability	Customer Satisfaction
Requirements & Planning	Prevents missing/incorrect features	Avoids costly late-stage rework	Product matches real customer needs
Design	Clean, consistent architecture	Handles traffic and failures gracefully	Smooth, intuitive user experience
Development	Well-structured, maintainable code	Fewer runtime errors in checkout/cart	Faster page loads, fewer glitches
Testing	Bugs caught before release	Payment & order flows proven stable	Fewer failed transactions
Deployment	Verified, stable release process	Minimal downtime at launch	Uninterrupted shopping access
Maintenance	Continuous code improvement	Long-term system stability	Ongoing trust and retention



Real-World Example: Amazon

Seeing SDLC applied in a platform used by millions daily

- **Product Search:** requirements defined precise filters and ranking rules; design chose a fast search architecture.
- **Cart:** development built real-time price and stock updates; testing verified cart accuracy under load.
- **Payment:** security testing protects transactions across multiple payment methods.
- **Order Tracking:** continuous maintenance keeps tracking accurate across couriers worldwide.
- Every phase of SDLC directly supports the **scale, trust, and speed** customers expect from Amazon.



Product Search



Cart



Payment



Order Tracking

Conclusion

- **SDLC turns** a vague e-commerce idea into a structured, buildable plan.
- Every phase — from requirements to maintenance — reduces risk and cost.
- The result is software that is **higher quality, more reliable, and trusted by customers**.
- For engineers, SDLC is not overhead — it is the **discipline that makes scale possible**.

Thank You

Questions Welcome